

ASSIGNMENT

2110413 COMPUTER SECURITY

Assignment 4
Fundamentals of Cryptography

Computer Engineering, Chulalongkorn University

Worralop Srichainont

Student ID: 6632200221

September 14, 2026

Contents

1	Part I: Cryptanalysis	1
1.1	Ciphertext	1
1.2	English Knowledge	1
1.2.1	Letter frequency	1
1.2.2	Short English words	1
1.3	Recovering the plaintext	2
1.3.1	Complete the substitution table	2
1.3.2	Result	4
1.4	Caesar brute-force program	4
2	Part II: Kasiski examination	6
3	Part III: Block-cipher modes on a bitmap	7
3.1	Procedure	7
3.2	Result and analysis	7
4	Part IV: Hashing, ciphers, and digital signatures	8
4.1	Experimental design	8
4.2	Measured results	8
4.3	Performance versus security	8
4.4	Digital signatures	9

1 Part I: Cryptanalysis

1.1 Ciphertext

The given ciphertext is a short English message written in a classical cipher:

PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR.
QDFP FP ZK LIU BROJZK MOLTROE.

1.2 English Knowledge

1.2.1 Letter frequency

Spaces and the period were ignored. There are 59 letters and 20 distinct symbols.

Letter	Count
P	7
R, O, F	6
Q	5
L	4
S, A, Z, K	3
C, D, J	2
X, I, U, B, M, T, E	1

Table 1: Letter frequencies in the assigned ciphertext.

The three most frequent ranks are **P** (7), then the tie **R, O, F** (6). In English, E, T, A, O, I, N, S, H, R are usually the most common letters, so P is a reasonable first guess for E or S.

1.2.2 Short English words

- Common two-letter words include OF, TO, IN, IS, IT, AS, and AN.
- Common three-letter words include THE, AND, FOR, and ARE.

In the ciphertext, **FP** appears twice. **QDFP FP** is a four-letter word followed by that same pair, which matches the English pattern **THIS IS**. Then, we take a look on **QDR** becomes **TH?**, so it is certain that $R \rightarrow E$.

Cipher letters	Shift	Plain Letters
Q	+3	T
D	+4	H
F	+3	I
P	+3	S
R	+13	E

Table 2: First guess of substitution letters

1.3 Recovering the plaintext

1.3.1 Complete the substitution table

We tried to use the substitution table above and got these:

Encrypted	Result	Guess
PRCSOFQX	SE???ITY	SECURITY
AFOPQ	?IRST	FIRST
CZSPR	C?USE	CAUSE
LA	OF	OF
JFPALOQSKR	?ISFORTU?E	MISFORTUNE
ZK	AN	AN
LIU	OLD	OLD
BROJZK	GERMAN	GERMAN
MOLTROE	PROVERB	PROVERB

Table 3: First guess of substitution letters

Therefore, we got the complete substitution table as

Cipher letters	Shift	Plain Letters
P	+3	S
R	+13	E
C	+0	C
S	+2	U
O	+3	R
F	+3	I
Q	+3	T
X	+1	Y
D	+4	H
A	+5	F
Z	+1	A
L	+3	O
J	+3	M
K	+3	N
I	+3	L
U	+9	D
B	+5	G
M	+3	P
T	+2	V
E	+23	B

Table 4: Complete substitution letters

1.3.2 Result

SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
THIS IS AN OLD GERMAN PROVERB.

Note

I used around 15 minutes to crack this cipher text manually.

1.4 Caesar brute-force program

The program tries all 26 shifts. It gives one point for each common English word, then prints the candidates from highest score to lowest.

```
caesar__bruteforce.py

1 import sys
2
3 WORDS = {"A", "AN", "THE", "IS", "OF", "TO", "IN", "THIS",
4         "SECURITY", "FIRST", "CAUSE", "MISFORTUNE"}
5 SAMPLE = "VHFXULWB LV WKH ILUVW FDXVH RI PLVIRUWXQH."
6
7
8 def decrypt(text, shift):
9     return "".join(
10         chr((ord(c) - ord("A") - shift) % 26 + ord("A"))
11         if c.isalpha() else c
12         for c in text.upper()
13     )
14
15
16 def score(text):
17     words = "".join(c if c.isalpha() else " " for c in text).split()
18     return sum(word in WORDS for word in words)
19
20
21 ciphertext = " ".join(sys.argv[1:]) or SAMPLE
22 results = []
23
24 for shift in range(26):
25     plaintext = decrypt(ciphertext, shift)
26     results.append((score(plaintext), shift, plaintext))
27
28 for points, shift, plaintext in sorted(
29     results, key=lambda row: (-row[0], row[1])):
30     print(f"shift={shift:2} score={points}: {plaintext}")
```

For the sample, the first result is:

Output

```
1 shift= 3 score=7: SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
```

The homework ciphertext does not fully decrypt with this program because it uses different shifts for different letters, rather than one Caesar shift.

2 Part II: Kasiski examination

Vigenère repeats a keyword, so each keyword position acts like a different Caesar cipher. Kasiski examination uses repeated ciphertext patterns to estimate the keyword length:

1. Find ciphertext strings of at least three letters that appear more than once.
2. Measure the distance between each pair of repeats.
3. Find a common factor of the distances. It is a likely keyword length.
4. Split the text by keyword position and use frequency analysis on each group.

After finding the length, the Vigenère cipher becomes several smaller Caesar ciphers, which can be cracked separately.

3 Part III: Block-cipher modes on a bitmap

3.1 Procedure

First, I resized the source image and converted it to a 1-bit binary PBM:

Image conversion

```
1 magick docs/images/source.jpg \  
2   -resize 2000x2000! -colorspace Gray -threshold 50% \  
3   pbm:org.pbm
```

I removed the two-line PBM header and encrypted only the 500,000-byte pixel body. This size is divisible by AES's 16-byte block size.

Encrypt the PBM body

```
1 openssl enc -aes-256-ecb -in org.x -out enc-ecb.x \  
2   -nosalt -nopad -K 001122...ccddeeff \  
3   \  
4 openssl enc -aes-256-cbc -in org.x -out enc-cbc.x \  
5   -nosalt -nopad -K 001122...ccddeeff \  
6   -iv 0102030405060708090a0b0c0d0e0f10
```

Finally, I added the original PBM header back to each encrypted body.

3.2 Result and analysis

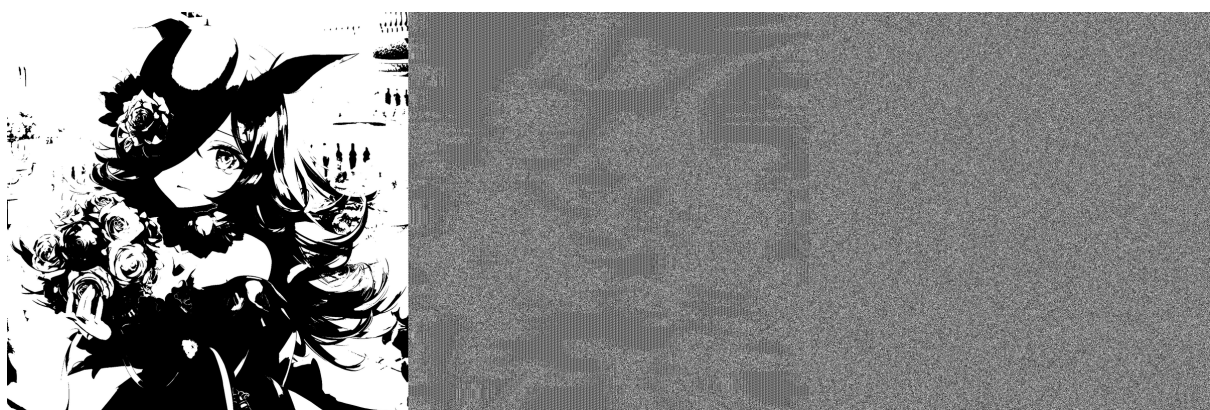


Figure 1: Original 1-bit image (left), AES-256-ECB (middle), AES-256-CBC (right).

ECB encrypts equal plaintext blocks into equal ciphertext blocks, so the picture's shape is still visible. CBC mixes each block with the previous encrypted block and an IV, so its result looks random. This shows why ECB should not be used for data with repeated patterns.

4 Part IV: Hashing, ciphers, and digital signatures

4.1 Experimental design

I used OpenSSL 3.6.4 on an Apple M1 Pro. Each test ran for 3 seconds and was repeated three times. The tables show the median result. RC4 and Blowfish need OpenSSL's legacy provider.

Commands used for Part 4(a)

```
1 openssl speed -seconds 3 sha1
2 openssl speed -seconds 3 -provider legacy -provider default rc4
3 openssl speed -seconds 3 -provider legacy -provider default blowfish
4 openssl speed -seconds 3 dsa
```

4.2 Measured results

Algorithm	16 B	64 B	256 B	1024 B	8192 B	16384 B
SHA-1	111625	391697	1092786	1830493	2316151	2333136
RC4	875828	1140693	1233343	1244972	1261681	1259033
Blowfish	121621	124320	125773	125994	126441	125495

Table 5: Median throughput in 1000s of bytes per second (OpenSSL units).

DSA size	sign (s)	verify (s)	sign/s	verify/s
1024 bit	0.000068	0.000053	14708	18846
2048 bit	0.000193	0.000173	5172	5770

Table 6: Median DSA sign and verify rates.

4.3 Performance versus security

- **SHA-1** is a fast hash, but collision attacks make it unsafe for new security systems. SHA-256 is a safer choice.
- **RC4** is fast, but its keystream has known biases, so it is obsolete.
- **Blowfish** is slower and has a small 64-bit block size. AES is preferred today.
- **DSA** is much slower because it uses public-key mathematics. It provides signatures, not bulk encryption.

4.4 Digital signatures

1. The sender hashes the message.
2. The sender signs the hash with their private key.
3. The receiver hashes the message and checks the signature with the sender's public key.

Hashing is fast and detects changes. Public-key signing is slower but proves who signed the message. Together they provide integrity, authentication, and non-repudiation without applying slow public-key operations to the whole message.